



Chapter 2

Array and Structures

数组与结构

Youxun Yao

Mobile Multimedia Convergence Network Lab,
Ajou University

目录

- 数组---Array
- 动态分配数组--Dynamically Allocated Arrays
- 结构体与联合体--Structures and Unions
- 多项式--Polynomials
- 稀疏矩阵--Sparse Matrices
- 字符串--Strings

数组：ADT

- 一段连续的内存[memory]（实践角度）
- 一组相同类型的数据集合
- 一个<索引, 值> [$\langle \text{index}, \text{value} \rangle$] 的集合
 - 每个索引 [index] 都有一个与之对应的值 [value]
- 数组的操作
 - 创建一个新数组
 - 检索一个值
 - 存储一个值

数组: ADT

❖ 数组的结构:

对象: 一个<索引, 值>对的集合, 其中每个索引都有一个值与之对应。

索引是一个一维或者多维的有限的有序集合。

例如, 一个一维数组的索引可以是 $\{0, \dots, n-1\}$, 二维数组的索引可以是 $\{(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)\}$, 以此类推

函数:

for all $A \in \text{Array}, i \in \text{index}, x \in \text{item}, j, \text{size} \in \text{integer}$

创建数组(j, list) ::= **return** 一个 j 维数组的 list , 在 j 元组中, 第 i 个元素表示第 i 维的大小, 数组中的元素 $[\text{Items}]$ 未被定义。

元素检索(A, i) ::= **if** ($i \in \text{index}$) **return** 返回数组 A 中与索引值关联的元素, **else return** 错误[error]

数组存储(A, i, x) ::= **if** ($i \in \text{index}$) **return** 一个插入了 $\langle i, x \rangle$ 对的与数组 A 相同的数组 **else return** error

end array

C语言中的数组

❖ 一维数组

➤ 含有5个整数的数组: `int list[5];`

➤ 含有5个整数的指针数组: `int *plist[5];`

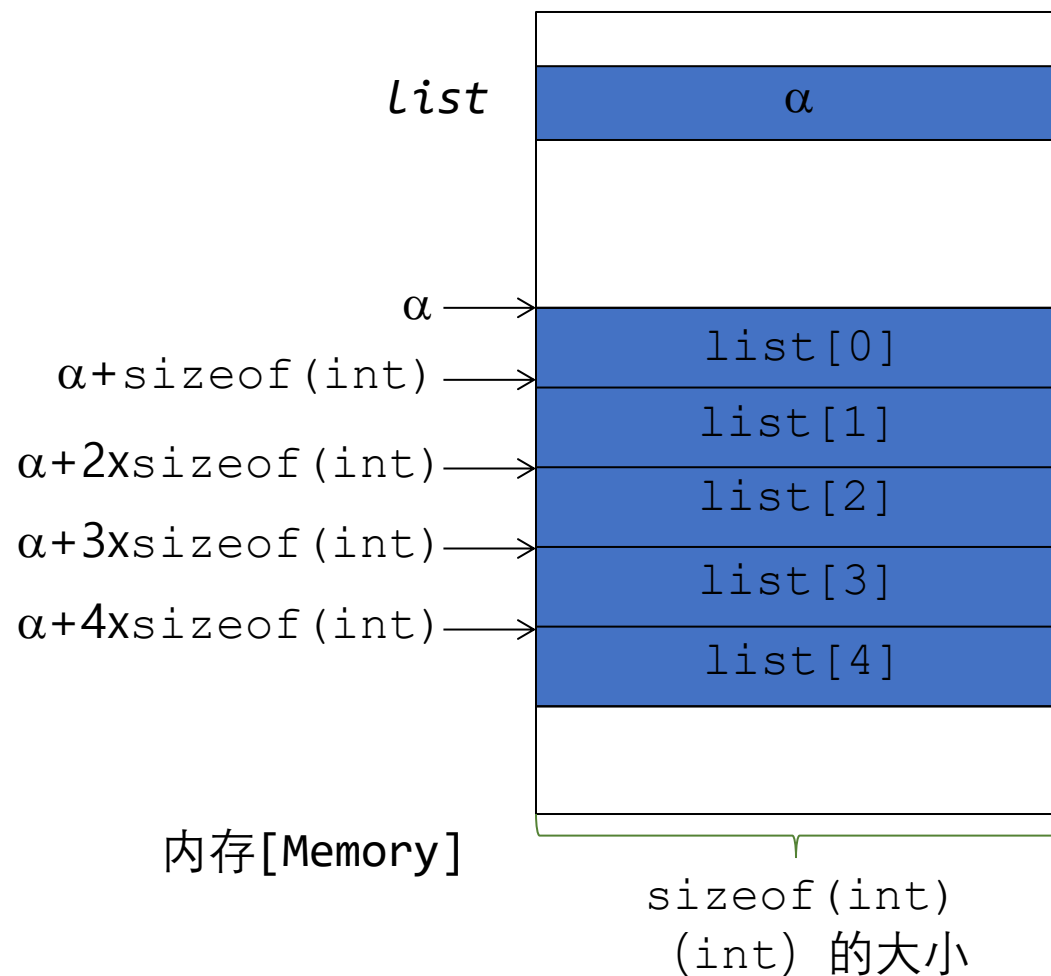
五个数组元素，每个元素包含一个指向整数的指针

C语言中的数组

```
int list[5];
```

变量[variable]	内存地址[Memory Address]
list[0]	base address = α
list[1]	$\alpha + \text{sizeof}(\text{int})$
list[2]	$\alpha + 2 \times \text{sizeof}(\text{int})$
list[3]	$\alpha + 3 \times \text{sizeof}(\text{int})$
list[4]	$\alpha + 4 \times \text{sizeof}(\text{int})$

$\text{list}[i] : \alpha + i \times \text{sizeof}(\text{int})$



示例

```
#define MAX_SIZE 100 [定义 MAX_SIZE 为100]
float sum(float [], int);
float input[MAX_SIZE], answer;
void main(void)
{
    int i;
    for (i = 0; i < MAX_SIZE; i++)
        input[i] = i;
    answer = sum(input, MAX_SIZE);      /* input = &input[0] */
    print("The sum is: %d\n", answer);
}

/* input [输入] 被复制到一个临时位置，并且与formal parameter list[形式参数列表]关联 */
float sum(float list[], int n)
{
    int i;
    float tempsum = 0;
    for (i = 0; i < n; i++)
        tempsum += list[i];
    return tempsum;
}
```

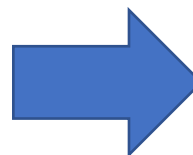
示例

❖ 一维数组寻址

编写一个函数，打印出数组中第*i*个元素的地址以及在该地址上的值[value]:

```
int one[] = {0, 1, 2, 3, 4};
```

```
void print1 (int *ptr, int rows )      /* 作为 print1( &one[0], 5 ) */
{
    /* print out a one-dimensional array using a pointer */
    int i;
    printf( "Address Contents\n" );
    for ( i=0; i < rows; i++ )
        printf( "%8u%5d\n", ptr+i, *(ptr+i) );
    printf( "\n" );
}
```



Address	Contents
12344868	0
12344872	1
12344876	2
12344880	3
12344884	4

动态分配数组

❖ 一维数组

```
#include <math.h>
#include <stdio.h>
#define MAX_SIZE 101

main()
{
    int i, n, list[MAX_SIZE];

    printf("Enter the number of number to generate: ");
    scanf("%d", &n);

    for (i=0;i<n;i++)
    {
        list[i]=rand()%1000;
        printf ("%d\n", list[i]);
    }
}
```

如果
数组大小未知会怎样?

动态分配数组

```
int i, n, *list;

printf("Enter the number of number to generate: ");
scanf("%d", &n);

if (n < 1) {
    fprintf(stderr, "Improper value of n \n");
    exit (EXIT_FAILURE);
}

list=malloc(n * sizeof(int));
if (list==NULL) {
    fprintf(stderr, "lack of memory\n");
    exit (EXIT_FAILURE);
}
```

延时这个决定到运行时

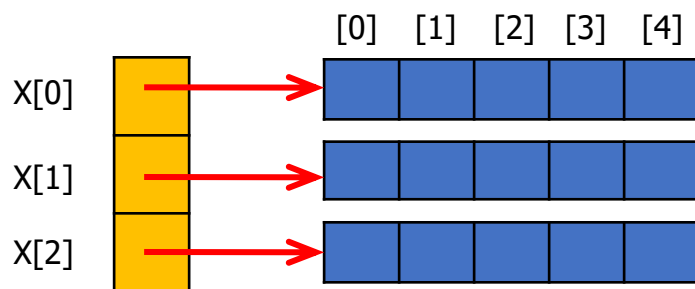
- 失败条件:
1. $n < 1$
 2. 没有足够的内存来存储数字列表

动态分配数组

❖ 二维数组

➤ 二维数组每个元素的本身可以看作一个一维数组

- `int x[3][5];`
 - `x` 的长度[length]为 3
 - `x`中的每个元素是一个长度为 5 的一维数组



Array-of-arrays representation
数组的数组表示方法

动态分配数组

```
int** make2dArray(int rows, int cols)
{
    /*创建一个二维 行×列 数组 */
    int** x, i;
    /*为行指针分配内存*/
    MALLOC(x, rows * sizeof (*x));

    /*为每一行分配内存*/
    for (i = 0; i < rows; i++)
        MALLOC(x[i], cols * sizeof (**x));
    return x;
}
```

```
void main()
{
    int** myArray;
    myArray = make2dArray(5, 10);
    myArray[2][4]=6;
}
```

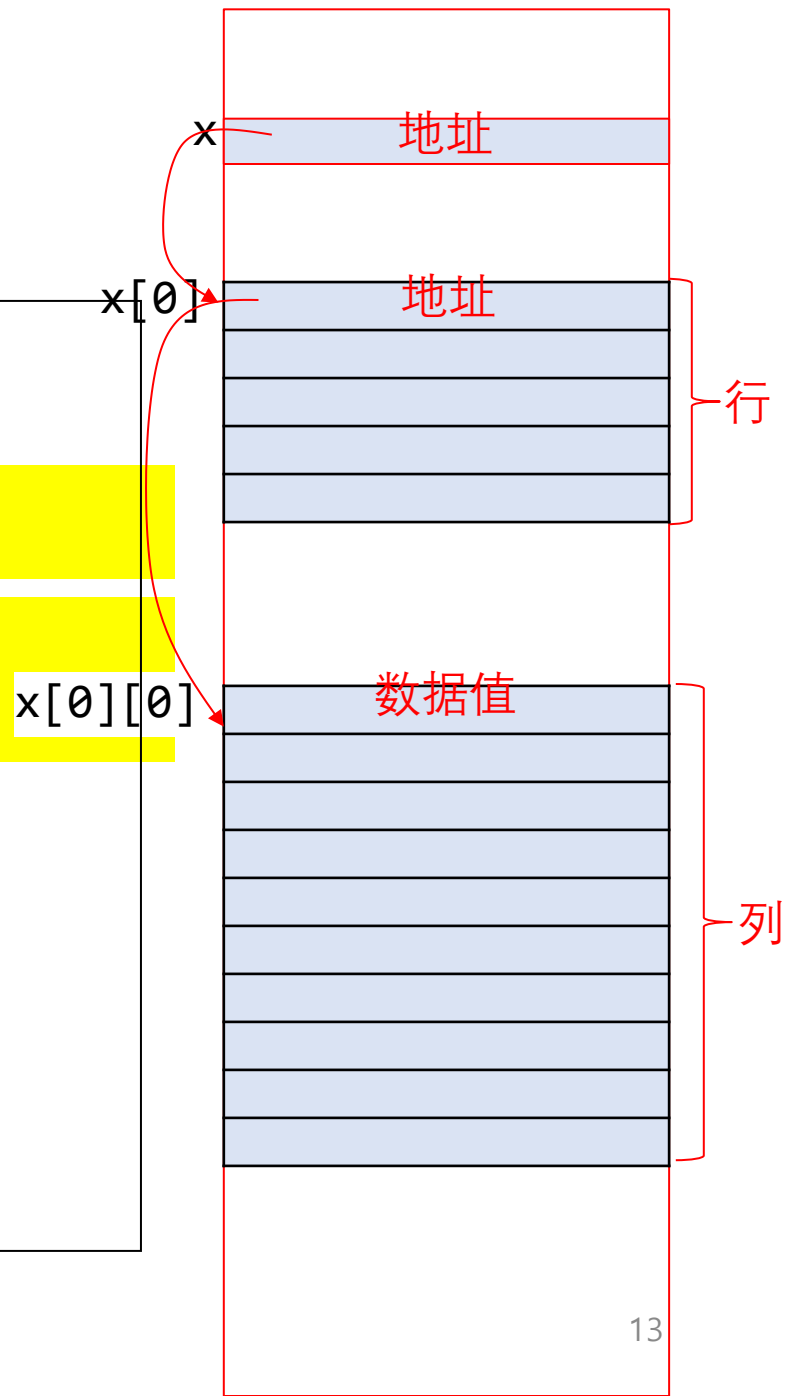
```
#define MALLOC(p,s) \
    if (!((p)=malloc(s))) {\
        fprintf(stderr, "Insufficient memory");\
        exit(EXIT_FAILURE);\
    } (Chapter 1, p.7)
```

动态分配数组

```
int** make2dArray(int rows, int cols)
{
    /*创建一个二维 行×列 数组 */
    int** x, i;
    /*为行指针分配内存*/
    MALLOC(x, rows * sizeof (*x));

    /*为每一行分配内存*/
    for (i = 0; i < rows; i++)
        MALLOC(x[i], cols * sizeof(**x));
    return x;
}

void main()
{
    int** myArray;
    myArray = make2dArray(5,10);
    myArray[2][4]=6;
}
```



结构体 (Structure)

- ❖ 数组 : 一组相同类型的数据集合
- ❖ 结构体 : 一种分组数据的替代方式
 - 允许数据的类型不同
 - 是一个数据项的集合
 - 每个数据项通过其类型和名称来标识
- ❖ 访问结构体
 - 结构体成员运算符: “.”

```
struct {  
    char    name[10];  
    int     age;  
    float   salary;  
} person;  
  
strcpy(person.name, "james");  
person.age=10;  
person.salary=35000;
```

结构体

❖ 创建数据类型: **typedef**

```
typedef struct humanBeing {  
    char    name[10];  
    int     age;  
    float   salary;  
} humanBeing;
```

or

```
typedef struct {  
    char    name[10];  
    int     age;  
    float   salary;  
} humanBeing;
```

❖ 声明变量

❖ humanBeing person1, person2;

If person1's birthday is May 28, 2021,
person1.dob.month = 5;
person1.dob.day = 28;
person1.dob.year = 2021;

❖ 在结构上嵌入结构

```
typedef struct {  
    int month;  
    int day;  
    int year;  
} date;
```

```
typedef struct humanBeing {  
    char name[10];  
    int age;  
    float salary;  
    date dob;  
} humanBeing;
```

联合体 (union)

- ❖ 与结构体相似
- ❖ 但是联合体的字段必须共享内存空间
 - 在任何给定时间，只有一个字段是活动的
- ❖ 示例 : 为 male 和 female 添加字段

```
#define TRUE 1
#define FALSE 0
typedef struct {
    enum tagField {female, male} sex;
    union {
        int children;
        int beard;
    } u;
} sexType;
```

```
typedef struct {
    char name[10];
    int age;
    float salary;
    date dob;
    sexType sexInfo;
} humanBeing;
```


联合体

```
#define TRUE 1
#define FALSE 0
typedef struct sexType {
    enum tagField {female, male} sex;
    union {
        int children;
        int beard;
    } u;
};
```

```
typedef struct humanBeing {
    char name[10];
    int age;
    float salary;
    date dob;
    sexType sexInfo;
};
```

```
humanBeing person1, person2;
```

```
person1.sexInfo.sex=male;
person1.sexInfo.u.beard=FALSE;
```

```
person2.sexInfo.sex = female;
person2.sexInfo.u.children = 4;
```

首先在标记字段中输入数值
以确定联合体中哪个字段处于活动状态

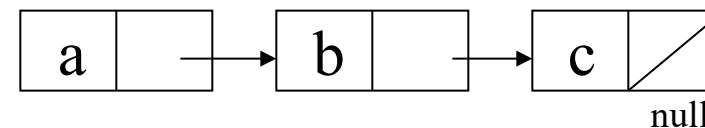
自引用结构体(Self-Referential Structures)

❖ 一个或者多个成员是指向自身的指针

```
typedef struct list* listPtr;  
typedef struct list {  
    char data;  
    listPtr link;  
};
```

```
list item1, item2, item3;
```

```
item1.data = 'a';  
item2.data = 'b';  
item3.data = 'c';  
item1.link = item2.link = item3.link = NULL;  
item1.link = &item2;  
item2.link = &item3;
```



有序列表(Ordered List)

❖ 有序列表(线性[linear]): ($item_0, item_1, item_2, \dots, item_{n-1}$)

❖ 示例:

- 一星期
 - (Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday)
- 衣服扑克牌
 - (Ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King)
- 美国参加第二次世界大战的年份
 - (1941, 1942, 1943, 1944, 1945)
- 瑞士参加第二次世界大战的年份
 - (): an empty list

有序列表的操作(Ordered List)

❖ 示例：

- ① 求列表的长度 n .
- ② 从左到右（或从右到左）阅读项目 $[item]$.
- ③ 读取第 i 个元素
- ④ 将新值存储到第 i 个位置
- ⑤ 在位置 i 插入一个新元素，使编号为 i 、 $i + 1$ 、...、 n 的元素变成编号为 $i + 1$ 、 $i + 2$ 、...、 $n + 1$ 的元素
- ⑥ 删除位置 i 上的元素，使编号为 $i + 1, \dots, n$ 的元素编号变为 $i, i + 1, \dots, n - 1$

• 数组：顺序映射

- 将列表元素 $item_i$ 与数组索引 i 关联
- 适用于①~④的操作
- 但⑤~⑥的运算却不是这样。
 - 这将引导我们在第 4 章中考虑有序列表的非顺序映射

多项式ADT [Polynomials ADT]

❖ 多项式

$$\bullet A(x) = 3x^2 + 2x + 4 \quad B(x) = x^4 + 10x^3 + 3x^2 + 1$$

❖ ADT 多项式

对于: $p(x) = a_1x^{e_1} + \dots + a_nx^{e_n}$;

一组有序的 $\langle e_i, a_i \rangle$ 对, 其中 a_i 在系数中, e_i 在指数中, e_i 是 ≥ 0 的整数

在函数中:

对于所有 $poly, poly1, poly2 \in \text{多项式}, coef \in \text{系数}, expon \in \text{指数}$

多项式	Zero()	::= return 多项式, $p(x) = 0$
布尔型	IsZero(poly)	::= if (poly) return FALSE else return TRUE
系数	Coef(poly, expon)	::= if (expon $\in$ poly) return its 系数 else return zero
指数	LeadExp(poly)	::= return poly 中的最大指数

多项式ADT [Polynomials ADT] (Cont.)

多项式附加 *Polynomial Attach*(*poly*, *coef*, *expon*) ::= **if** (*expon* $\in$ *poly*) **return** error
else return the polynomial *poly*
with the term $\langle \textit{coef}, \textit{expon} \rangle$ inserted

多项式移除 *Polynomial Remove*(*poly*, *expon*) ::= **if** (*expon* $\in$ *poly*)
return the polynomial *poly* with the
term whose exponent is *expon* deleted
else return error

多项式单项相乘 *Polynomial SingleMult*(*poly*, *coef*, *expon*) ::= **return** the polynomial
 $\textit{poly} \cdot \textit{coef} \cdot x^{\textit{expon}}$

多项式相加 *Polynomial Add* (*poly1*, *poly2*) ::= **return** the polynomial
 $\textit{poly1} + \textit{poly2}$

多项式相乘 *Polynomial Mult*(*poly1*, *poly2*) ::= **return** the polynomial
 $\textit{poly1} \cdot \textit{poly2}$

End *Polynomial*

多项式表示法 1

❖按指数递减的顺序存储系数

```
#define MAX_DEGREE 101
typedef struct {
    int degree;
    float coef[MAX_DEGREE];
} polynomial;
```

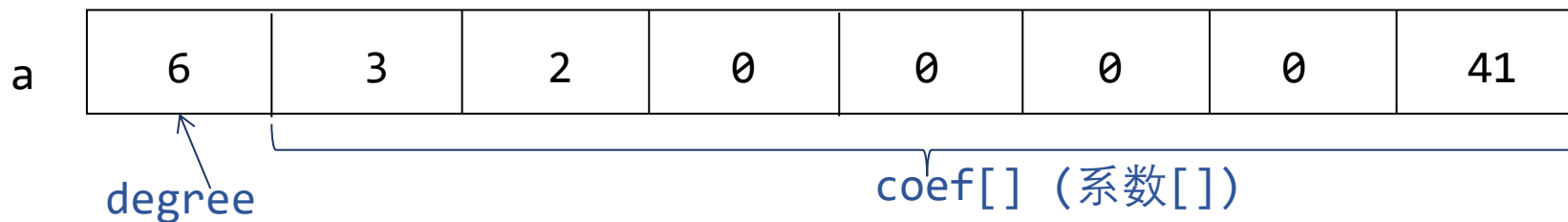
多项式表示法 1

❖ 示例:

$$A(x) = 3x^6 + 2x^5 + 41 = 3x^6 + 2x^5 + 0x^4 + 0x^3 + 0x^2 + 0x^1 + 41x^0$$

polynomial a;

a.degree = n, a.coef[i] = a_{n-i} , $0 \leq i \leq n$



How about $2x^{1000} + 1$?

- 为大多数操作提供非常简单的算法
- 在多项式的项稀疏情况(即大多数系数为0)下浪费大量空间

多项式表示法 2

❖使用全局数组来存储多项式

```
#define MAX_TERMS 100
typedef struct {
    float coef;
    int expon;
} polynomial;
polynomial terms[MAX_TERMS];
```

多项式表示法 2

❖ 示例:

$A(x) = 2x^{1000} + 1$ and $B(x) = x^4 + 10x^3 + 3x^2 + 1$

	<i>startA</i>	<i>finishA</i>	<i>startB</i>			<i>finishB</i>	<i>avail</i>
	↓	↓	↓			↓	↓
<i>Coef</i>	2	1	1	10	3	1	
<i>exp</i>	1000	0	4	3	2	0	

- *startA* , *startB* : A 和 B 的第一项的索引
- *finishA*, *finishB* : A 和 B 的最后一项的索引
 - *avail* : 数组中下一个空闲位置的索引
- 存储要求: *start*, *finish*, 2*(*finish*-*start*+1)
- 当所有项都不为零时: 空间是第一个项的两倍

多项式加法

```
void padd (int startA, int finishA, int startB, int finishB, int *startD, int *finishD)
{
    /*将 A(x) 和 B(x) 相加, 得到 D(x) */
    float coefficient;
    *startD = avail;
    while ( startA <= finishA && startB <= finishB )
        switch ( COMPARE(terms[startA].expon, terms[startB].expon) )
        {
            case -1:          /* a expon < b expon */
                attach( terms[startB].coef, terms[startB].expon );
                startB++;
                break;
            case 0:           /*等指数*/
                coefficient = terms[startA].coef + terms[startB].coef;
                if ( coefficient )
                    attach ( coefficient, terms[startA].expon );
                startA++;
                startB++;
                break;
            case 1:           /* a expon > b expon */
                attach( terms[startA].coef, terms[startA].expon );
                startA++;
        }
    /* end of switch */
}
```

多项式加法

```
/* 添加 A(x) 的剩余项*/
for( ; startA <= finishA; startA++ )
{
    attach( terms[startA].coef, terms[startA].expon );
}
/* 添加 B(x) 的剩余项*/
for( ; startB <= finishB; startB++ )
{
    attach( terms[startB].coef, terms[startB].expon );
}
*finishD =avail -1;
}
```

❖ padd 函数

- A 和 B 中非零项的数量是影响时间复杂度的最重要因素
- 时间复杂度 $O(n+m)$
 - m: A 中非零项的数量
 - n: A 中非零项的数量

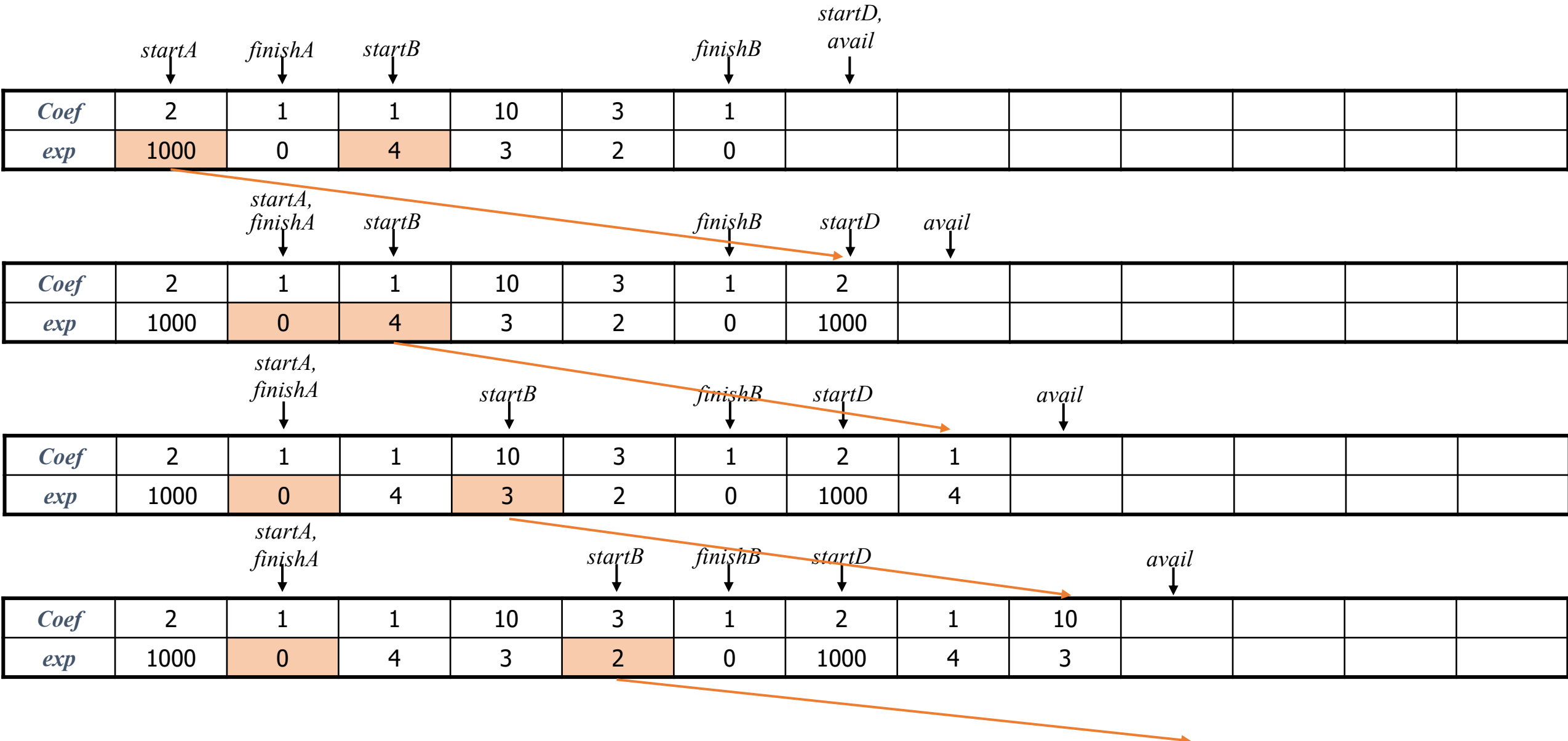
```
void attach(float coefficient, int exponent)
{
    /*在多项式中加入新的项*/
    if (avail >= MAX_TERMS)
    {
        fprintf(stderr, "Too many terms in the polynomial\n");
        exit(EXIT_FAILURE);
    }
    terms[avail].coef = coefficient;
    terms[avail++].expon = exponent;
}
```

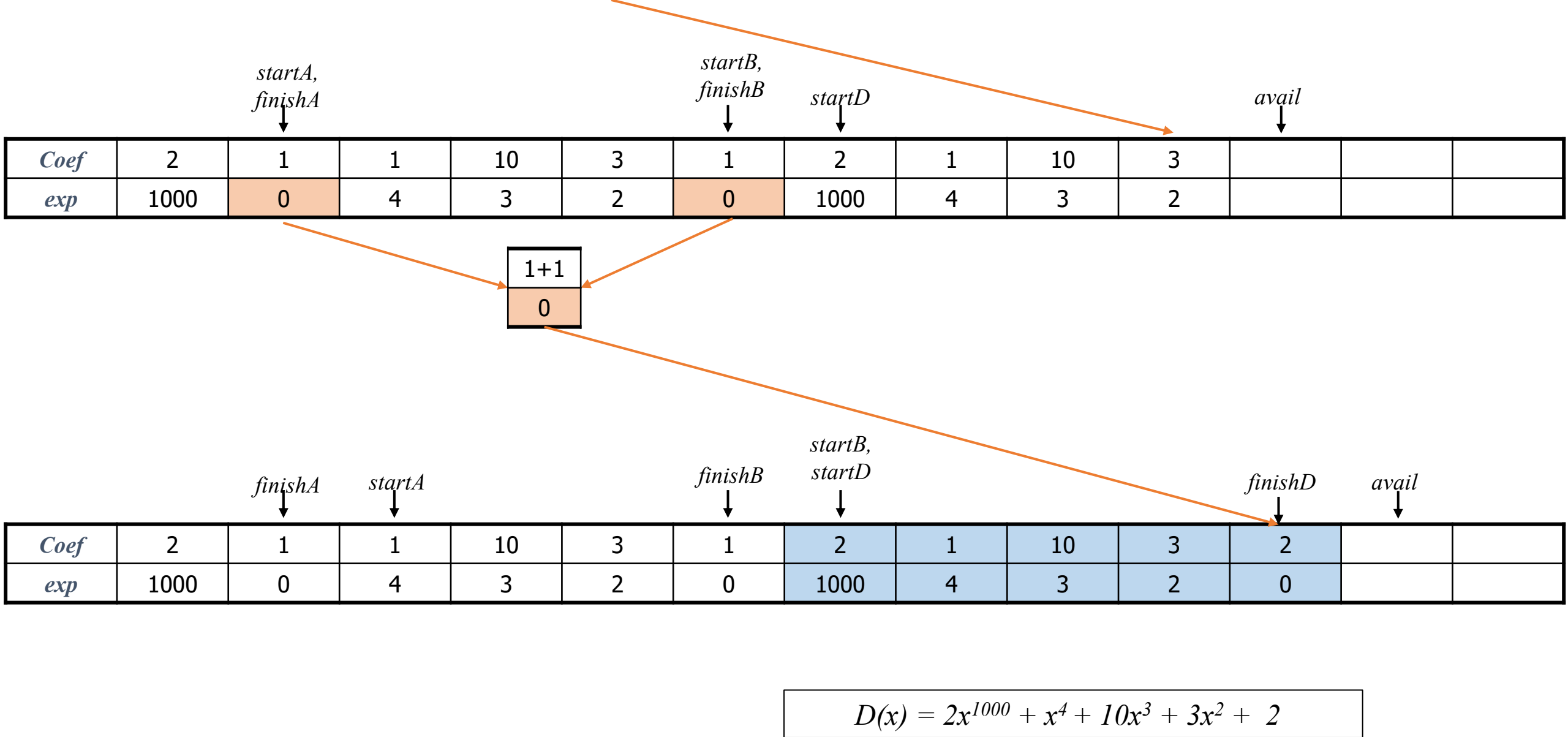
```
#define MAX_TERMS 100
typedef struct {
    float coef;
    int expon;
} polynomial;
polynomial terms[MAX_TERMS];
```

❖ Problem

- 当不再需要多项式时需要压缩 [Compaction]
- 数据移动 (data movement) 需要时间

$A(x) = 2x^{1000} + 1,$
 $B(x) = x^4 + 10x^3 + 3x^2 + 1$





稀疏矩阵 (sparse matrix)

❖ 一个包含 m 行 n 列元素的矩阵，如图所示

	Col0	Col1	Col2
Row0	-27	3	4
Row1	6	82	-2
Row2	109	-64	11
Row3	12	8	9
Row4	48	27	47

	Col0	Col1	Col2	Col3	Col4	Col5
Row0	15	0	0	22	0	-15
Row1	0	11	3	0	0	0
Row2	0	0	0	-6	0	0
Row3	0	0	0	0	0	0
Row4	91	0	0	0	0	0
Row5	0	0	28	0	0	0

稀疏矩阵 (sparse matrix)

❖ 稀疏矩阵

- 一个包含大量零元素 [many zero entries] 的矩阵
- 将稀疏矩阵表示为二维数组
 - 浪费空间

	Col0	Col1	Col2
Row0	-27	3	4
Row1	6	82	-2
Row2	109	-64	11
Row3	12	8	9
Row4	48	27	47

	Col0	Col1	Col2	Col3	Col4	Col5
Row0	15	0	0	22	0	-15
Row1	0	11	3	0	0	0
Row2	0	0	0	-6	0	0
Row3	0	0	0	0	0	0
Row4	91	0	0	0	0	0
Row5	0	0	28	0	0	0

稀疏矩阵 ADT

❖ 结构体 [Structure] *SparseMatrix*

对于：一个三元组的集合， $\langle row, column, value \rangle$, 其中，行和列都是整数，构成唯一组合，值[value]来自集合项[item]

函数：

对于所有 $a, b \in SparseMatrix, x \in item, i, j, maxCol, maxRow \in index$

创建稀疏矩阵 *SparseMatrix* Create($maxRow, maxCol$) ::=

return 一个稀疏矩阵，最多可以存储 $maxItems = maxRow \times maxCol$ 个元素，其最大行数为 $maxRow$, 最大列数为 $maxCol$

转置稀疏矩阵 *SparseMatrix* Transpose(a) ::=

return 将每个三元组的行值和列值互换后产生的矩阵

稀疏矩阵 ADT

稀疏矩阵相加 *SparseMatrix* Add(*a*, *b*) ::=

if *a* and *b* 的维度相同

return 将相应项目（即行列值相同的项目）相加所产生的矩阵

else return error

稀疏矩阵相乘 *SparseMatrix* Multiply(*a*, *b*) ::=

if *a* 中的列数等于 *b* 中的行数

return 由*a*和*b*相乘得出的*d*矩阵，根据公式：

$$d[i][j] = \sum (a[i][k] \cdot b[k][j])$$

其中 $d(i, j)$ 是第 (i, j) 元素

else return error

稀疏矩阵

```
SparseMatrix Create(maxRow, maxCol) ::=  
    #define MAX_TERMS 101          /* 最大项+1*/  
    typedef struct {  
        int col;  
        int row;  
        int value;  
    } term;  
    term a[MAX_TERMS];
```

❖ 稀疏矩阵表达方法

- 每个元素通过三元组<row, col, value> [<行, 列, 值>]来表征

	Col0	Col1	Col2	Col3	Col4	Col5
Row0	15	0	0	22	0	-15
Row1	0	11	3	0	0	0
Row2	0	0	0	-6	0	0
Row3	0	0	0	0	0	0
Row4	91	0	0	0	0	0
Row5	0	0	28	0	0	0

	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

行、列和各自值的数量

按行和
行内按列排序

示例

	Col0	Col1	Col2
Row0	0	3	0
Row1	0	0	-2
Row2	0	0	0
Row3	0	8	0
Row4	48	0	0

	行数	列数	值数
	row	col	value
$a[0]$	5	3	4
$a[1]$	0	1	3
$a[2]$	1	2	-2
$a[3]$	3	1	8
$a[4]$	4	0	48

大致内存需求

❖ 500 x 500 矩阵, 1994 个非零元素, 每个元素 4 字节 [bytes]

2D 数组: $500 \times 500 \times 4 = 1\text{M bytes} = 1000\text{K bytes}$

三元一维数组: $3 \times 1995 \times 4 \approx 23\text{K bytes}$

转置矩阵 [Transposing matrix]

- 转置[Transpose]: 交换行列

1) for 每一行 i

- 将三元组 $\langle i, j, value \rangle$ 存储为转置矩阵的三元组 $\langle j, i, value \rangle$
- 遗憾的是, 不知道该把元素放在哪里 $\langle j, i, value \rangle$
 - 直到我们处理完前面的所有元素为止
- 示例

$(0, 0, 15) \implies (0, 0, 15)$

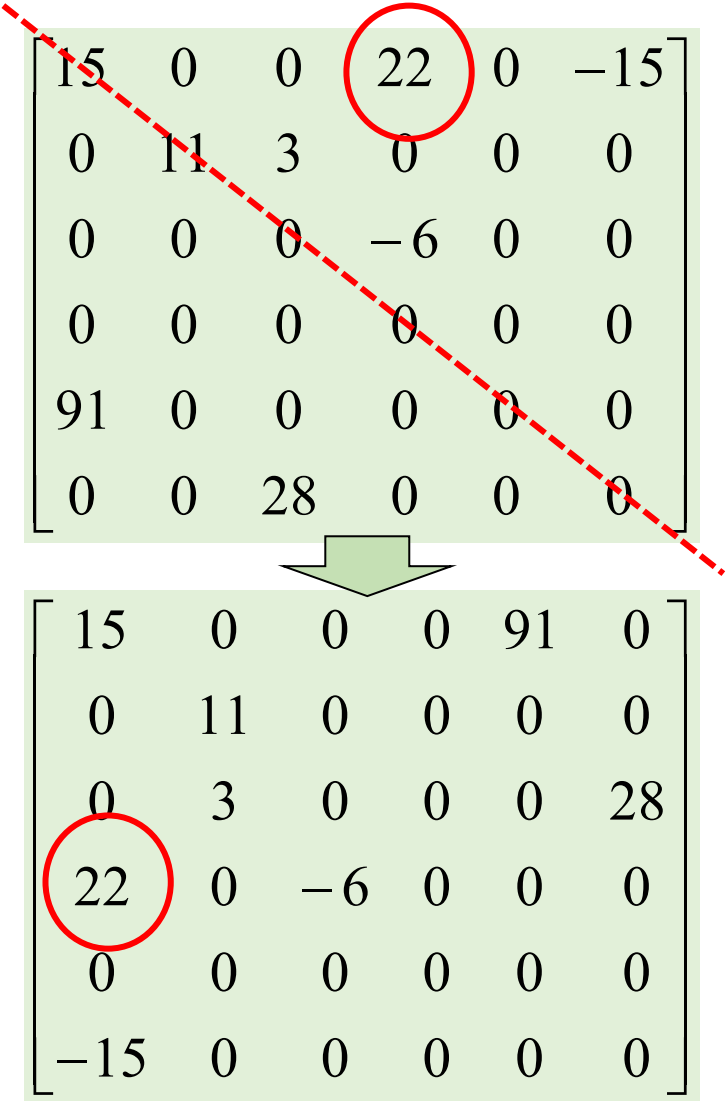
$(0, 3, 22) \implies (3, 0, 22)$

$(0, 5, -15) \implies (5, 0, -15)$

2) for j 列的每个元素,

- 将元素 $\langle i, j, value \rangle$ 置于 $\langle j, i, value \rangle$ 的位置
- $O(columns \times elements)$
 - 扫描数组 $|columns|$ 次, 且数组有 $|elements|$ 个元素。

转置矩阵 [Transposing matrix]



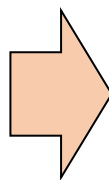
	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	3	0	22
a[3]	5	0	-15
a[4]	1	1	11
a[5]	2	1	3
a[6]	3	2	-6
a[7]	0	4	91
a[8]	2	5	28

转置矩阵 [Transposing matrix]

for 每一行 i
将三元组 $\langle i, j, value \rangle$ 存储为转置矩阵的三元组 $\langle j, i, value \rangle$

	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

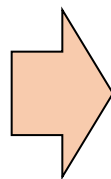


	row	col	value	
b[0]				
b[1]				
b[2]				
b[3]	a[??]	3	0	22
b[4]				
b[5]				
b[6]				
b[7]				
b[8]				

转置矩阵 [Transposing matrix]

for j 列的每个元素,
将元素 $\langle i, j, value \rangle$ 置于 $\langle j, i, value \rangle$ 的位置

	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28



	row	col	value
b[0]	6	6	8
b[1]	0	0	15
b[2]	0	4	91
b[3]	1	1	11
b[4]	2	1	3
b[5]	2	5	28
b[6]	3	0	22
b[7]	3	2	-6
b[8]	5	0	-15

转置矩阵 [Transposing matrix]

```
void transpose(term a[], term b[])
{
    /* b is set to the transpose of a*/
    int n, i, j, currentb;
    n = a[0].value; /*元素总数*/
    b[0].row = a[0].col; /* b中的行数等于a中的列数*/
    b[0].col = a[0].row; /* b中的列数等于a中的行数*/
    b[0].value = n;
    if (n > 0) { /* 非零矩阵*/
        currentb = 1;
        for (i=0; i<a[0].col; i++){ /* 列转置*/
            for (j = 1; j <= n; j++){ /* 扫描每个元素*/
                if (a[j].col == i){
                    b[currentb].row = a[j].col;
                    b[currentb].col = a[j].row;
                    b[currentb].value = a[j].value;
                    currentb++;
                } /* end of if */
            } /* end of for j */
        } /* end of for i */
    } /* end of if */
}
```

→ 0 (*columns · elements*)

快速矩阵变换 (Fast transpose matrix)

	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	3
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

0

22

行 0 中有 2 个元素
行 1 中有 1 个元素
行 2 中有 2 个元素

	row	col	value
b[0]	6	6	8
b[1]	0	0	15
b[2]			
b[3]			
b[4]			
b[5]			
b[6]	3	0	22
b[7]			
b[8]			

快速矩阵变换 (Fast transpose matrix)

- Step 1: 计算转置矩阵中每行的非零元素数量，等于原始矩阵中每列的非零元素数量。
- Step2: 计算转置矩阵中每行的起始位置，等于前面所有行的非零元素数量之和。
- Step 3: 将原始矩阵中的非零元素按顺序移动到转置矩阵中。

原矩阵

	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

转置矩阵中每一行的非零
元素数量

转置矩阵中每一行的
起始位置

rowTerms

[0]	2
[1]	1
[2]	2
[3]	2
[4]	0
[5]	1

startingPos

[0]	1
[1]	3
[2]	4
[3]	6
[4]	8
[5]	8

```
startingPos[0] = 1;
for(i = 1; i < num_cols; i++){
    startingPos[i] =
        startingPos[i-1] +
        rowTerms[i-1];
}
```

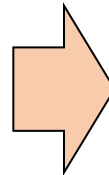
a[2]: <0,3,22> → b[startingPos[3]]: <3,0,22>

```
for(i = 1; i <= numTerms; i++) {
    j = startingPos[a[i].col]++;
    b[j].row=a[i].col;    b[j].col=a[i].row;  b[j].value=a[i].value;
}
```

	[0]	[1]	[2]	[3]	[4]	[5]
startingPos	3	4	6	8	8	9

原矩阵

	row	col	value
a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28



转置矩阵

	row	col	value
b[0]	6	6	8
b[1]	0	0	15
b[2]	0	4	91
b[3]	1	1	11
b[4]	2	1	3
b[5]	2	5	28
b[6]	3	0	22
b[7]	3	2	-6
b[8]	5	0	-15

→ 0 (columns + elements)

快速矩阵变换

```
void fastTranspose(term a[], term b[])
{
    int rowTerms[MAX_COL], startingPos[MAX_COL];
    int i,j, numCols = a[0].col, numTerms = a[0].value;
    b[0].row = numCols; b[0].col = a[0].row;
    b[0].value = numTerms;
    if (numTerms > 0) { /* 非零矩阵 */
        for(i = 0; i < numCols; i++) { rowTerms[i] = 0; }
        for(i=1; i<= numTerms; i++){rowTerms[a[i].col]++;}
        startingPos[0] = 1;
        for(i = 1; i < numCols; i++){
            startingPos[i]=startingPos[i-1]+rowTerms[i-1];
        }
        for(i = 1; i <= numTerms; i++) {
            j = startingPos[a[i].col]++;
            b[j].row = a[i].col; b[j].col = a[i].row;
            b[j].value = a[i].value;
        }
    }
}
```

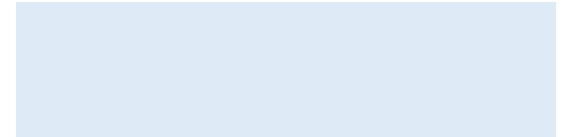
→ 0 (columns + elements)

字符串 (string) ADT

- ❖ *String* [字符串] 是由零个或多个字符组成的有限集合
- ❖ C 语言中的*String*表示法

```
#define MAX_SIZE 100  
char s[MAX_SIZE] = { “dog” };
```

s[0]	s[1]	s[2]	s[3]
d	o	g	\0

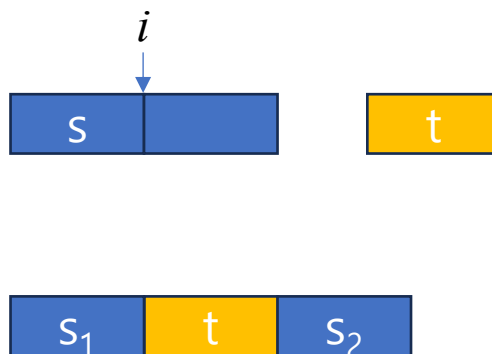


❖ C 字符串函数

- `char *strcat(char *dest, char *src)`
- `char *strncat(char *dest, char *src, int n)`
- `int strcmp(char *str1, char *str2)`
- `int strncmp(char *str1, char *str2, int n)`
- `char *strcpy(char *dest, char *src)`
- `char *strncpy(char *dest, char *src, int n)`
- `size_t strlen(char *s)`
- `char *strchr(char *s, int c)`
- `char *strtok(char *s, char *delimiters )`
- `char *strstr(char *s, char *pat)`
- `size_t strspn(char *s, char *spanset)`
- `size_t strcspn(char *s, char *spanset)`
- `char *strpbrk(char *s, char *spanset)`

字符串插入

❖ 从字符串 s 的 i 位置开始插入字符串 t



字符串插入

❖ 从字符串 s 的 i 位置开始插入字符串 t

```
void strnins( char *s, char *t, int i )
{
    char string[MAX_SIZE], *temp = string;
    if ( i < 0 && i > strlen(s) ) {
        fprintf( stderr , “ 位置超出边界 \n” );
        exit(1);
    }
    if ( !strlen(s) )
        strcpy( s, t );                /*将 t 复制到 s 中*/
    else if ( strlen(t) ) {
        strncpy( temp, s, i ); /*将 i 个字符从 s 复制到 temp */
        strcat( temp, t );
        strcat( temp, (s+i) );
        strcpy( s, temp );
    }
}
```

s →

a	m	o	b	i	l	e	\0
---	---	---	---	---	---	---	----

t →

u	t	o	\0
---	---	---	----

temp →

\0

strncpy(temp, s, i); char string[MAX_SIZE], *temp=string;

temp →

a	\0
---	----

strcat(temp, t);

temp →

a	u	t	o	\0
---	---	---	---	----

strcat(temp, s+i);

temp →

a	u	t	o	m	o	b	i	l	e	\0
---	---	---	---	---	---	---	---	---	---	----

strcpy(s, temp);

s →

a	u	t	o	m	o	b	i	l	e	\0
---	---	---	---	---	---	---	---	---	---	----

```
void main()
{
    char s[MAX_SIZE]="amobile";
    char t[MAX_SIZE]="uto";
    strnins(s,t,1);
    printf("%s\n", s);
    return;
}
```

模式匹配 (Pattern Matching)

```
#include <string.h>
...
if (t=strstr(string, pat))
    printf("The string from strstr is : %\n", t);
else
    printf("The pattern was not found with strstr\n");
```

❖ 假设我们有两个字符串： *string* 和 *pat*，其中 *pat* 是要在字符串中搜索的模式

❖ 最简单的方法

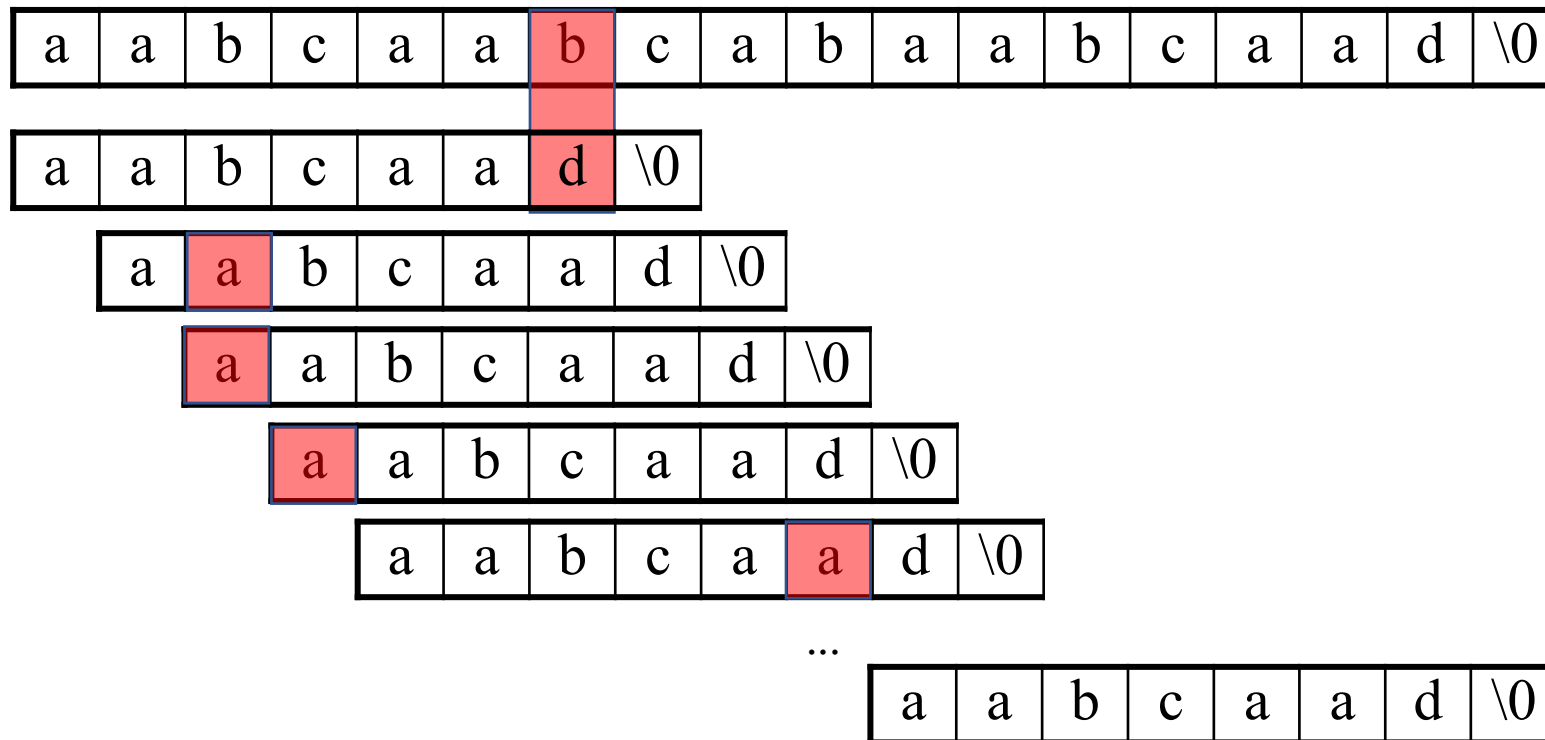
- 使用内置函数 `strstr(char *s, char *pat)`
 - 返回一个在 *string* 中 *pat* 的其实位置的指针
 - 如果 *pat* 不在 *string* 中，返回一个空指针

❖ 开发我们自己的模式匹配功能

- 最简单但效率最低的方法是顺序检查字符串中的每个字符，直到找到模式或到达字符串末尾。
 - 如果模式不在字符串中，时间复杂度为 $O(nm)$ ，其中 n 表示模式的长度， m 表示字符串的长度

模式匹配 (Pattern Matching)

❖ 朴素方法 [Naïve approach]



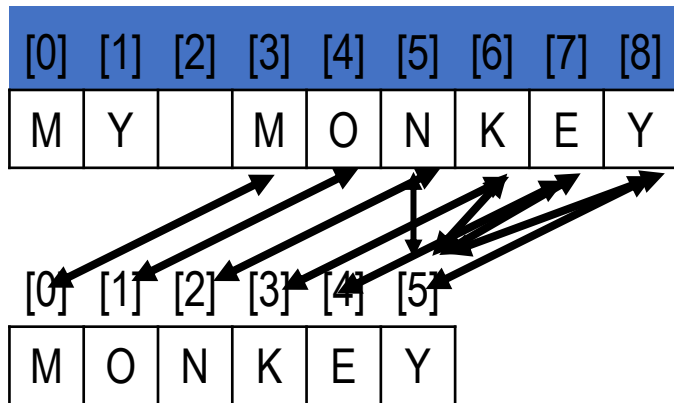
nfind

❖改进点

- 当 $\text{strlen}(pat)$ 大于 $string$ 中剩余字符的数量时，则退出
- 先检查 pat 和 $string$ 第一个和最后一个字符，再检查剩余字符

```
int nfind(char *s, char *pat)
{
    int i, j, start = 0, lasts = strlen(s)-1, lastp = strlen(pat)-1;
    int endmatch = lastp;
    for (i = 0; endmatch <= lasts; endmatch++, start++) {
        if (s[endmatch] == pat[lastp])
            for (j = 0, i = start; j < lastp && s[i] == pat[j]; i++, j++) ;
        if (j == lastp)
            return start;
    }
    return -1;
}
```

- ❖ $nfind$ 的最差计算时间为 $O(nm)$, 其中 n 是 pat 的长度, m 是 $string$ 的长度



```

lasts=8
lastp=5
endmatch= 8
start= 0
i=0
j=0

```

```

int i, j, start = 0
int lasts = strlen(string)-1, lastp = strlen(pat)-1;
int endmatch = lastp;

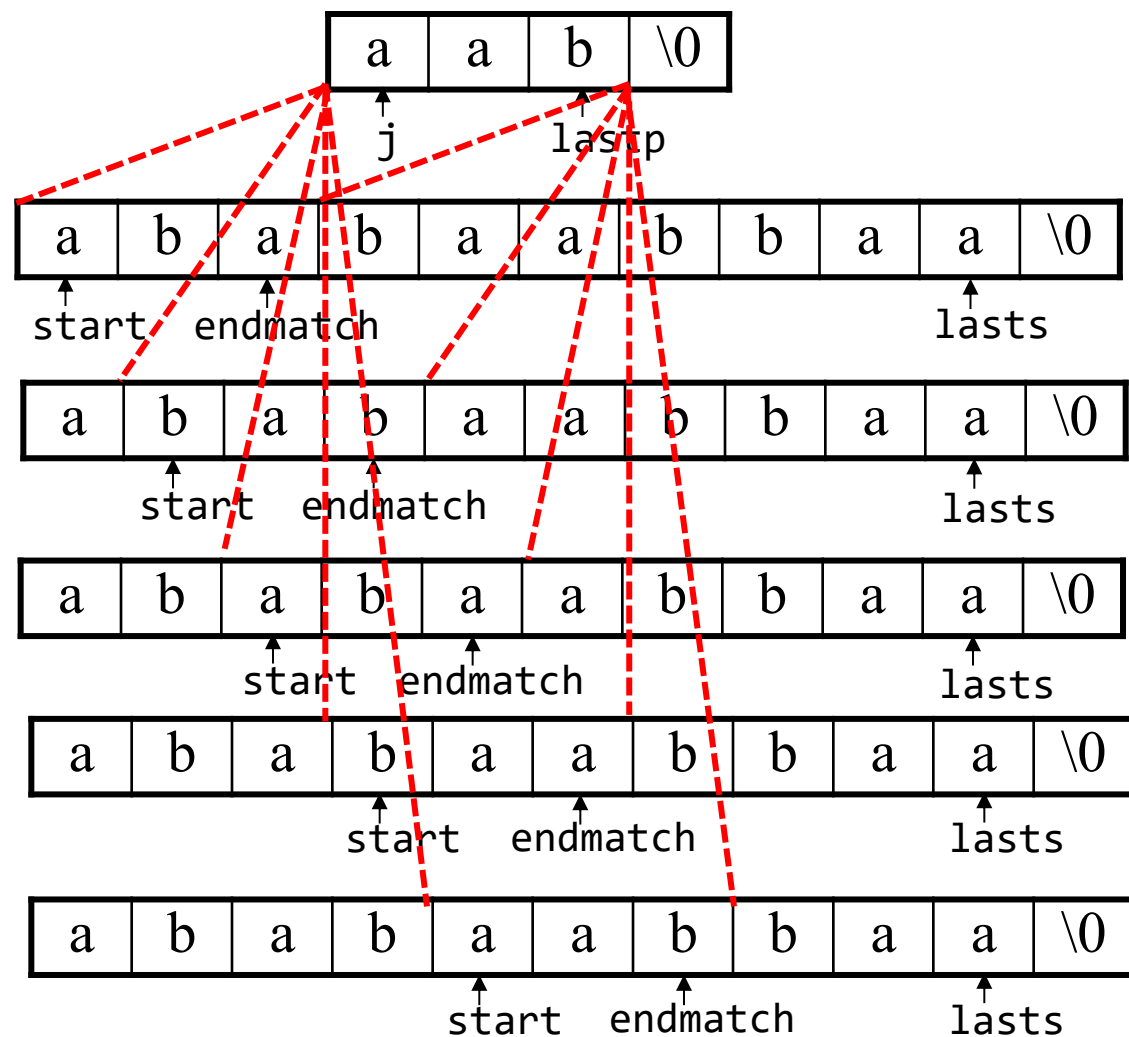
```

```

for (i = 0; endmatch<=lasts; endmatch++, start++) {
    if (string[endmatch] == pat[lastp]){
        for (j=0, i=start;
            j<lastp && string[i]==pat[j]; i++, j++);
        if (j == lastp)
            return start;
    }
}

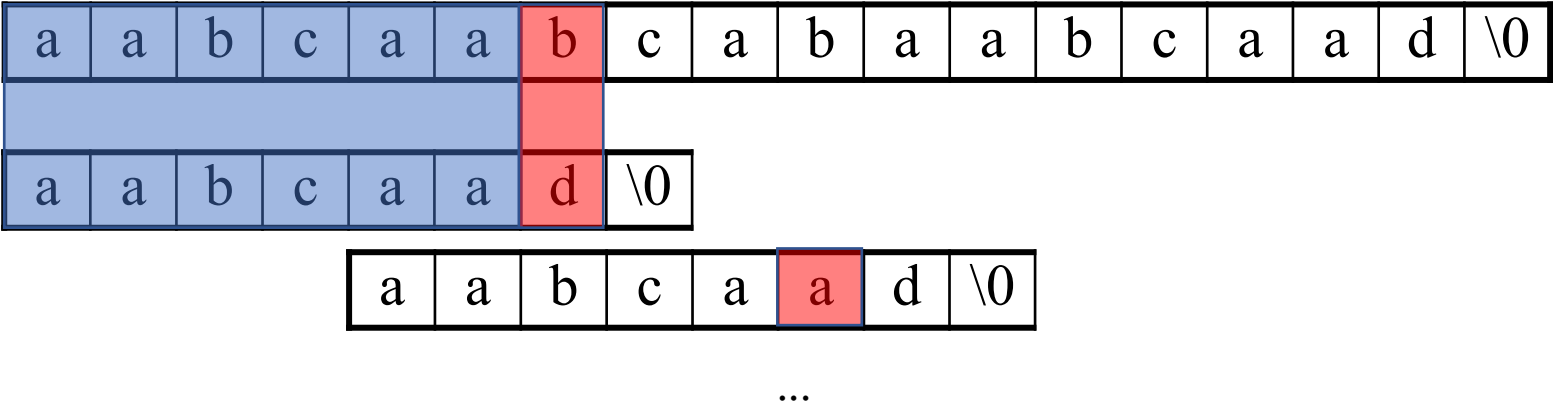
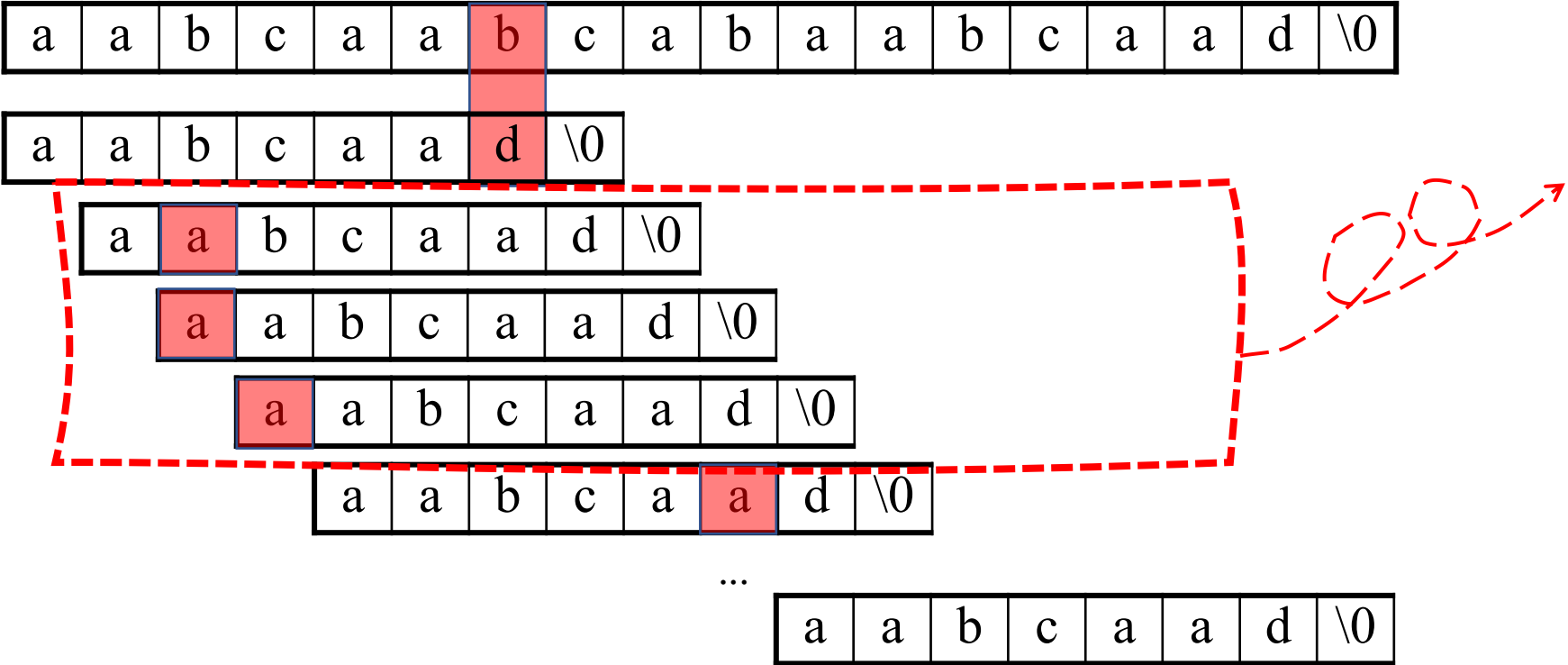
```

示例

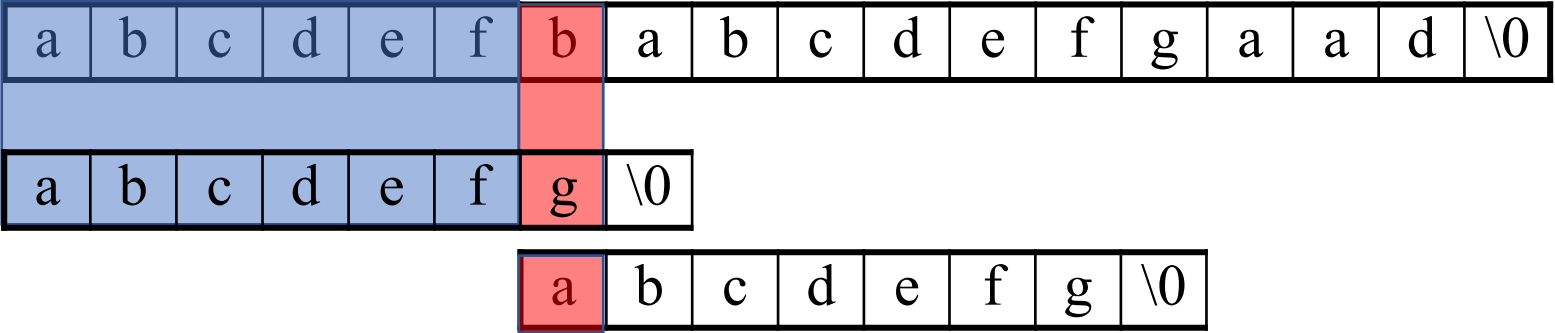
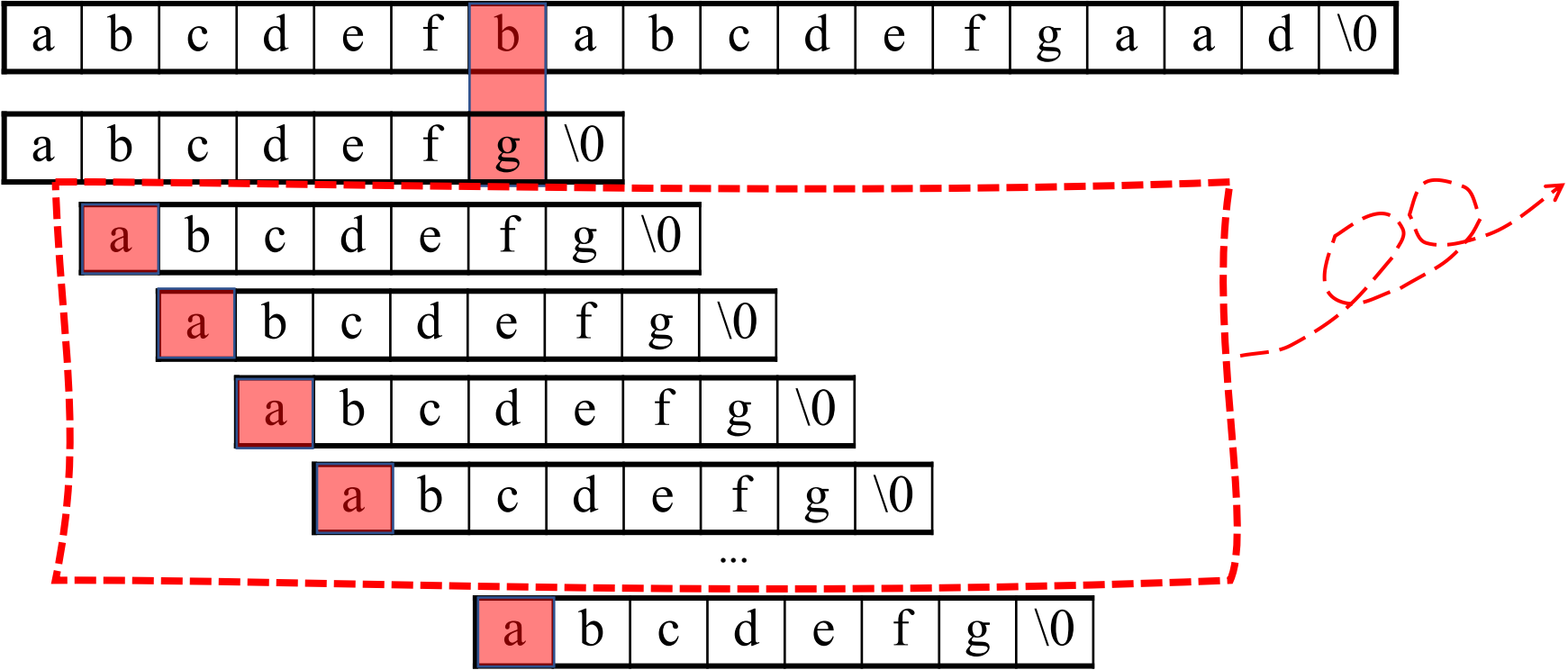


→ $O(\text{string 长度} \cdot \text{pattern 长度})$

Naïve approach



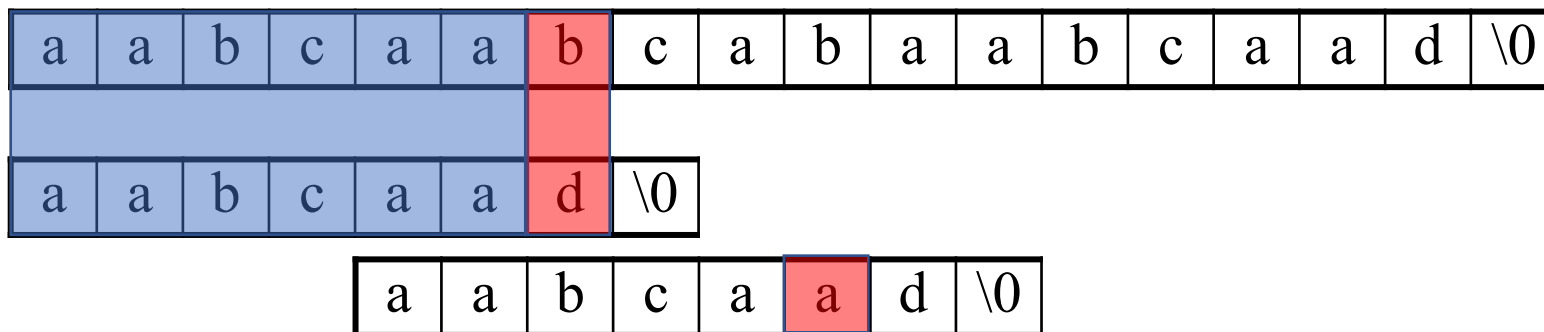
Naïve approach



Knuth, Morris, and Pratt algorithm

Knuth-Morris-Pratt 算法，简称 KMP算法， 是一种高效的字符串匹配算法， 由高德纳 (Knuth)， 莫里斯 (Morris)， 普拉特 (Pratt) 共同推出

❖ Key idea: 利用之前的匹配信息!



...

KMP 算法

❖ 示例:

- $pat = \text{'a b c a b c a c a b'}$
- $s = s_0 s_1 \cdots s_{m-1}$
- 确定是否在 s_i 处有一个匹配开始

❖ 如果 $s_i \neq a$, 继续比较 s_{i+1} 和 a

❖ 如果 $s_i = a$, 且 $s_{i+1} \neq b$ then, 执行比较 s_{i+1} 和 a 的操作

❖ 如果 $s_i s_{i+1} = ab$ 且 $s_{i+2} \neq c$ then, 查找匹配执行比较 pat 中的第一个字符和 s_{i+2}

❖ 如果出现先如下状况 $s_i s_{i+1} s_{i+2} s_{i+3} = abca$ 且 $s_{i+4} \neq b$:

$s = \text{'~ a b c a ? ? . . . ?'}$

$pat = \text{'a b c a b c a c a b'}$

- 查找匹配可以执行比较 s_{i+4} 和 pat 中 第二个字符 b 。

KMP 算法

❖ 定义

- 给定任意模式字符串 $p = p_0p_1 \dots p_{n-1}$ 该模式的失效函数 (*failure function*) $f(x)$ 定义如下

$$f(x) = \begin{cases} \max (i < j), & p = p_0p_1 \dots p_{n-1} \text{ 且 } i \geq 0 \\ -1, & \text{其他} \end{cases}$$

- Example

j	0	1	2	3	4	5	6	7	8	9
pat	a	b	c	a	b	c	a	c	a	b
f	-1	-1	-1	0	1	2	3	-1	0	1

❖ 规则

1. 部分匹配时恢复

如果出现部分匹配, 即 $s_{i-j} \dots s_{i-1} = p_0p_1 \dots p_{j-1}$ 且 $s_i \neq p_j$ 时, 可以通过比较 s_i 和 $p_{f(j-1)+1}$ 恢复匹配。

2. 完全失配时的恢复

如果 $j = 0$, 则可以通过比较 $s_i + 1$ 和 p_0 来恢复匹配。

失效函数示例

- Pattern : abcabcacab (n = 10) $p=p_0p_1\cdots p_9$, $p_0=a$, $p_9=b$

$$f(x) = \begin{cases} \max (i < j), & p = p_0p_1 \dots p_{n-1} \text{ 且 } i \geq 0 \\ -1, & \text{其他} \end{cases}$$

j	$f(j)$	Partial string	$f(j)$ results
0		a	-1
1	$p_0=p_1 (i = 0)$	ab	-1
2	$p_0 = p_2 (i = 0)$ $p_0p_1 = p_1p_2 (i = 1)$	abc	-1
3	$p_0 = p_3 (i = 0)$ $p_0p_1 = p_2p_3 (i = 1)$ $p_0p_1p_2 = p_1p_2p_3 (i = 2)$	abca	0
4	$p_0 = p_4 (i = 0)$ $p_0p_1 = p_3p_4 (i = 1)$ $p_0p_1p_2 = p_2p_3p_4 (i = 2)$ $p_0p_1p_2p_3 = p_1p_2p_3p_4 (i = 3)$	abcab	1

失效函数示例

- Pattern : abcabcacab (n = 10) $p=p_0p_1\cdots p_9$, $p_0=a$, $p_9=b$

$$f(x) = \begin{cases} \max (i < j), & p = p_0p_1 \dots p_{n-1} \text{ 且 } i \geq 0 \\ -1, & \text{其他} \end{cases}$$

j	$f(j)$	Partial string	$f(j)$ results
5	$p_0 = p_5 (i = 0)$ $p_0p_1 = p_4p_5 (i = 1)$ $p_0p_1p_2 = p_3p_4p_5 (i = 2)$ $p_0p_1p_2p_3 = p_2p_3p_4p_5 (i = 3)$ $p_0p_1p_2p_3p_4 = p_1p_2p_3p_4p_5 (i = 4)$	abcabc	2
6	$p_0 = p_6 (i = 0)$ $p_0p_1 = p_5p_6 (i = 1)$ $p_0p_1p_2 = p_4p_5p_6 (i = 2)$ $p_0p_1p_2p_3 = p_3p_4p_5p_6 (i = 3)$ $p_0p_1p_2p_3p_4 = p_2p_3p_4p_5p_6 (i = 4)$ $p_0p_1p_2p_3p_4p_5 = p_1p_2p_3p_4p_5p_6 (i = 5)$	abcbca	3

失效函数示例

- Pattern : abcabcacab (n = 10) $p=p_0p_1\cdots p_9$, $p_0=a$, $p_9=b$

$$f(x) = \begin{cases} \max (i < j), & p = p_0p_1 \dots p_{n-1} \text{ 且 } i \geq 0 \\ -1, & \text{其他} \end{cases}$$

j	$f(j)$	Partial string	$f(j)$ results
7	$p_0 = p_7 (i = 0)$ $p_0p_1 = p_6p_7 (i = 1)$ $p_0p_1p_2 = p_5p_6p_7 (i = 2)$ $p_0p_1p_2p_3 = p_4p_5p_6p_7 (i = 3)$ $p_0p_1p_2p_3p_4 = p_3p_4p_5p_6p_7 (i = 4)$ $p_0p_1p_2p_3p_4p_5 = p_2p_3p_4p_5p_6p_7 (i = 5)$ $p_0p_1p_2p_3p_4p_5p_6 = p_1p_2p_3p_4p_5p_6p_7 (i = 6)$	abcabcac	-1
8	$p_0 = p_8 (i = 0)$ $p_0p_1 = p_7p_8 (i = 1)$ $p_0p_1p_2 = p_6p_7p_8 (i = 2)$ $p_0p_1p_2p_3 = p_5p_6p_7p_8 (i = 3)$ $p_0p_1p_2p_3p_4 = p_4p_5p_6p_7p_8 (i = 4)$ $p_0p_1p_2p_3p_4p_5 = p_3p_4p_5p_6p_7p_8 (i = 5)$ $p_0p_1p_2p_3p_4p_5p_6 = p_2p_3p_4p_5p_6p_7p_8 (i = 6)$ $p_0p_1p_2p_3p_4p_5p_6p_7 = p_1p_2p_3p_4p_5p_6p_7p_8 (i = 7)$	abcabcaca	0

失效函数示例

• Pattern : abcabcacab (n = 10) $p=p_0p_1\cdots p_9$, $p_0=a$, $p_9=b$

$$f(x) = \begin{cases} \max (i < j), & p = p_0p_1 \dots p_{n-1} \text{ 且 } i \geq 0 \\ -1, & \text{其他} \end{cases}$$

j	f(j)	Partial string	f(j) results
9	$p_0 = p_9 (i = 0)$ $p_0p_1 = p_8p_9 (i = 1)$ $p_0p_1p_2 = p_7p_8p_9 (i = 2)$ $p_0p_1p_2p_3 = p_6p_7p_8p_9 (i = 3)$ $p_0p_1p_2p_3p_4 = p_5p_6p_7p_8p_9 (i = 4)$ $p_0p_1p_2p_3p_4p_5 = p_4p_5p_6p_7p_8p_9 (i = 5)$ $p_0p_1p_2p_3p_4p_5p_6 = p_3p_4p_5p_6p_7p_8p_9 (i = 6)$ $p_0p_1p_2p_3p_4p_5p_6p_7 = p_2p_3p_4p_5p_6p_7p_8p_9 (i = 7)$ $p_0p_1p_2p_3p_4p_5p_6p_7p_8 = p_1p_2p_3p_4p_5p_6p_7p_8p_9 (i = 8)$	abcabcacab	1

KMP算法

```
int pmatch( char *string, char *pat )
{
    /* KMP字符串匹配算法*/
    int i=0, j=0;
    int lens = strlen( string );
    int lenp = strlen( pat );
    while( i<lens && j<lenp ) {
        if( string[i] == pat[j] ) {
            i++;
            j++;
        }
        else if( j==0 ) i++;
        else j=failure[j-1]+1;
    }
    return ( (j==lenp) ? (i-lenp) : -1 );
}
```

```
void fail( char *pat )
{
    /* 计算pattern的故障函数*/
    int n=strlen( pat );
    failure[0] = -1;
    for( j=1; j<n; j++ ) {
        i=failure[j-1];
        while( (pat[j] != pat[i+1]) && (i >= 0) )
            i = failure[i];
        if( pat[j] == pat[i+1] )
            failure[j] = i+1;
        else
            failure[j] = -1;
    }
}
```

KMP 算法

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	...								
a	a	b	c	a	a	b	c	a	b	a	a	b	c	a	a	d	\0	
a	a	b	c	a	a	d	\0											
										a	a	b	c	a	a	d	\0	

...

```
while( i<lens && j<lenp ) {
    if( string[i] == pat[j] ) {
        i++;
        j++;
    }
    else if( j==0 ) i++;
    else      j=failure[j-1]+1;
}
```

故障函数

[0]	[1]	[2]	[3]	[4]	[5]	[6]
a	a	b	c	a	a	d
-1	0	-1	-1	0	1	-1

```
When i=6 and j=6,  
    string[6] != pat[6] (b!=d)  
j=f[6-1]+1;  
=f[5]+1 = 1+1 = 2;
```

```
i=6,j=2
Compare string[6] and pat[2] (b =b)
and continue
```

Question

str	a	b	c	d	e	f	b	a	b	c	d	e	f	g	a	a	d	\0
pat	a	b	c	d	e	f	g	\0										